



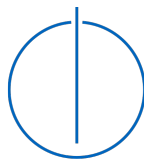
SCHOOL OF COMPUTATION
INFORMATION AND TECHNOLOGY
INFORMATICS

TECHNICAL UNIVERSITY OF MUNICH

Bachelor's Thesis in Informatics

**On-Policy Self-Distillation for Adaptive
Compute**

Yll Kryeziu





SCHOOL OF COMPUTATION
INFORMATION AND TECHNOLOGY
INFORMATICS

TECHNICAL UNIVERSITY OF MUNICH

Bachelor's Thesis in Informatics

**On-Policy Self-Distillation for Adaptive
Compute**

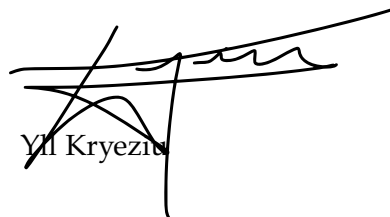
**On-Policy-Selbstdestillation für adaptiven
Rechenaufwand**

Author:	Yll Kryeziu
Supervisor:	Prof. Dr. Stefan Bauer
Advisor:	Prof. Dr. Stefan Bauer
Submission Date:	02.03.2026



I confirm that this bachelor's thesis in informatics is my own work and I have documented all sources and material used.

Munich, 02.03.2026



Ylli Kryeziu

Acknowledgments

I would like to express my heartfelt gratitude to Prof. Dr. Stefan Bauer for providing me with the opportunity to work on this topic. His guidance and the resources provided were instrumental in enabling this research.

A special thanks to Mihir, Franz and Alfred for the immensely helpful mentoring and guidance, countless discussions, proposed ideas, insightful feedback, and for making my first proper research experience so enjoyable.

Last but not least, I want to thank my parents and my sister for their never-ending support and love. You are my main source of motivation.

Abstract

By spending more computation at inference time, reasoning models have achieved significant performance gains in several domains. However, they often suffer from overthinking: models generate excessively long reasoning steps or redundant explanations for a problem, particularly simple ones, without improving the accuracy of the final answer. This thesis studies a self-contained approach to more adaptive reasoning by incentivizing a model to spend less compute on easy problems, while retaining compute expenditure for harder ones. We let a model rewrite its own reasoning traces to a length that is appropriate for the respective problem difficulty, and distill this behavior back into the model via token-level on-policy self-distillation.

To evaluate rewriting beyond token counts, we introduce a benchmark that compares original and rewritten traces across correctness, compression, structural efficiency, reflection quality, step-level recall, and precision against reference key steps generated by a more capable judge. On a 1,500-instance mathematics subset of OpenThoughts-114k and Qwen3 models from 1.7B to 14B parameters, self-refinement yields large reductions in reasoning trace length (about 78–83%, from roughly 6–8k to roughly 1.2–1.4k tokens) while preserving, and in most cases improving, final-answer correctness and maintaining high step-level quality.

We report results of distilling this behavior into the model. We show that our method leads to token usage reductions of 40–60% with no accuracy drop on a hold-out validation set. However, our preliminary results indicate a slight drop in performance on downstream LLM math benchmarks. Distillation reliably shortens reasoning traces (about 28–46% fewer tokens) and the downstream performance gap between baseline and distilled checkpoints narrows with model scale, indicating a more favorable length–robustness trade-off for larger models. Overall, the results show that self-refinement is a strong test-time compression operator and that on-policy self-distillation provides a promising path toward self-contained, adaptive-compute reasoning.

Kurzfassung

Durch den Einsatz zusätzlicher Rechenleistung zur Inferenzzeit haben Reasoning-Modelle in mehreren Domänen deutliche Leistungsgewinne erzielt. Gleichzeitig leiden sie häufig unter Overthinking: Sie erzeugen übermäßig lange Reasoning-Traces oder redundante Erklärungen, insbesondere bei einfachen Aufgaben, ohne dass sich die Genauigkeit der finalen Antwort verbessert. Diese Arbeit untersucht einen in sich geschlossenen Ansatz für adaptiveres Reasoning, indem ein Modell dazu gebracht wird, bei leichten Aufgaben weniger Rechenbudget zu verbrauchen und das notwendige Budget für schwierigere Aufgaben zu behalten. Dazu lassen wir das Modell seine eigenen Reasoning-Traces auf eine zur Problemschwierigkeit passende Länge umschreiben und destillieren dieses Verhalten anschließend über tokenbasierte On-Policy-Selbstdestillation zurück in das Modell.

Um das Umschreiben über reine Tokenanzahl hinaus zu bewerten, führen wir einen Benchmark ein, der originale und umgeschriebene Traces hinsichtlich Korrektheit, Kompression, struktureller Effizienz, Reflexionsqualität sowie schrittweisem Recall und Precision gegenüber Referenz-Schlüsselschritten vergleicht, die von einem leistungsfähigeren Judge-Model erzeugt werden. Auf einem 1,500-Instanzen-Mathematik-Subset von OpenThoughts-114k und für Qwen3-Modelle von 1,7B bis 14B Parametern führt Same-Model-Refinement zu starken Reduktionen der Tracelänge (ca. 78–83 %, von etwa 6–8k auf etwa 1,2–1,4k Tokens), während die Korrektheit der Antworten erhalten bleibt und sich in den meisten Fällen sogar verbessert, bei gleichzeitig hoher Qualität auf Schrittebene.

Wir berichten außerdem Ergebnisse zur Destillation dieses Verhaltens in das Modell. Wir zeigen eine Reduktion des Tokenverbrauchs um 40–60 % ohne Genauigkeitsverlust auf dem Validation-Set, jedoch weisen unsere vorläufigen Resultate auf LLM-Math-Benchmarks einen leichten Performance-Rückgang auf. Dennoch verkürzt die Distillation die Traces zuverlässig (ca. 28–46 % weniger Trace-Tokens), und die Erfolgsratenlücke zwischen Baseline und destillierten Checkpoints verringert sich mit wachsender Modellgröße. Dies deutet auf einen günstigeren Trade-off zwischen Länge und Robustheit bei größeren Modellen hin. Insgesamt zeigen die Ergebnisse, dass Same-Model-Rewriting ein starker Testzeit-Kompressionsoperator ist und dass On-Policy-Selbst-Distillation einen vielversprechenden Weg zu selbständigem, adaptivem Compute-Budgeting beim Reasoning eröffnet.

Contents

Acknowledgments	iii
Abstract	iv
Kurzfassung	v
1. Introduction	1
2. Background	4
2.1. Problem Setup	4
2.2. Token Probabilities and Distillation Objectives	4
2.2.1. Token Log-Probabilities Under a Prefix	4
2.2.2. Supervised Fine-Tuning	4
2.2.3. Knowledge Distillation	5
2.2.4. Self-Distillation and Context Distillation	5
2.3. On-Policy Distillation	6
2.4. Reasoning Traces and Rewriting	7
2.4.1. Redundancy and Compression	7
2.4.2. Distribution and Format Mismatch	7
2.4.3. Rewrite Benchmarking	8
3. Related Work	9
3.1. Reasoning Models and RL-Incentivized Long-Form Reasoning	9
3.1.1. RL post-training teaches models to “think longer”	9
3.1.2. Overthinking and Wasted Compute in Reasoning	9
3.2. Test-Time Compute Scaling	9
3.2.1. Prompting, sampling, search, and refinement-style reasoning	9
3.2.2. Adaptive Inference Policies and Early Stopping	10
3.3. Compression, Pruning, and Length Control	10
3.3.1. Prompt- and decoding-level length control	10
3.3.2. Training-Time Approaches for Conciseness	11
3.4. Evaluating Reasoning Traces and Thinking Efficiency	11
3.5. Self-Improvement, Rewriting, and Distillation	11
4. Method	12
4.1. Rewriting Benchmark	12
4.1.1. Pipeline and Artifacts	12

4.1.2.	Metrics and Extraction	13
4.1.3.	Failure Modes and Diagnostics	13
4.2.	On-Policy Rewrite and Distillation Loop	14
4.2.1.	Concrete Prompt and Context Construction	14
4.2.2.	Teacher Log-Probabilities on Student Tokens	15
4.2.3.	Training Loop	16
5.	Experiments	17
5.1.	Experimental Setup	17
5.1.1.	Datasets and sampling	17
5.1.2.	Used models and decoding settings	17
5.2.	Benchmark Results	17
5.2.1.	Discussion	18
5.3.	On-Policy Self-Distillation Training and Results	19
5.3.1.	Training	19
5.3.2.	Training Configuration	19
5.3.3.	Downstream Evaluation	20
6.	Conclusion	23
6.1.	Summary	23
6.2.	Limitations and Outlook	23
A.	Prompt Templates	24
A.1.	Code availability	30
	List of Figures	31
	List of Tables	32
	Bibliography	33

1. Introduction

Large language models (LLMs) can improve performance on reasoning tasks by producing intermediate reasoning steps before the final answer. Chain-of-Thought (CoT) prompting is a simple and widely used way of eliciting such step-by-step outputs, and has been shown to improve results on arithmetic, commonsense, and symbolic reasoning benchmarks [1]. Reasoning models such as Deepseek R1 [2] and OpenAI o1 [3] are incentivized to reason in natural language through supervised fine-tuning and reinforcement learning (RL), producing ever longer reasoning chains in order to maximize accuracy. At the same time, generating longer traces increases inference cost and latency, since each additional token requires computation.

Recent work suggests that longer reasoning is not always beneficial. In particular, models that tend to generate long traces allocate substantial compute to simple problems and produce redundant steps with little accuracy gain, a phenomenon described as overthinking [4]. From an efficiency perspective, this creates a mismatch between where computation is spent and where it is actually needed. This motivates methods that adapt the test-time reasoning effort to the difficulty of the input rather than always producing a long trace, even if not required for the problem.

A naive approach is to prompt the model to be concise when answering. However, systematic studies [5] of prompt-based compression indicate a persistent trade-off between reasoning length and accuracy, arguing that an appropriate compression level can vary across instances. This points to the need for methods that do not simply enforce global brevity, but instead aim to remove unnecessary tokens while preserving key steps required to tackle any given problem. Beyond global “be concise” prompting, recent work targets instance-adaptive efficiency and length-aware post-training. One line of work allocates per-input reasoning budgets, either by explicitly conditioning on a token budget and estimating that budget from difficulty [6], or by keeping a multi-step structure while making each intermediate step extremely concise [7]. This aligns with empirical evidence that the optimal compression level varies by instance [5]. A complementary line of work makes brevity part of the optimization objective by using length-controlled RL to train models to satisfy a user-specified reasoning-length constraint while preserving correctness [8]. Similarly, another approach [9] constructs preference pairs that reward short and correct responses over long and correct ones while explicitly guarding against underthinking. They further prune redundant backtracking by rewriting solutions using a larger model to only keep the first correct sub-solution and an additional check (FCS+1), and apply preference optimization (SimPO) [10] to encourage shorter sequences.

While these directions improve efficiency, many are not *self-contained* and rely on additional signals or components beyond a single model and its input. For example, token-budget

allocation typically requires an auxiliary notion of instance difficulty to map inputs to budgets [6]. Length-controlled RL and related objectives require a way to score correctness under different length constraints, often via reference ground-truth labels, verifiers, or reward models [8]. Preference-based approaches may require curated preference pairs using a stronger teacher model to generate preferred solutions [9]. Such dependencies make methods less general as they assume task-specific supervision or more capable models during training. They often cannot be applied in settings where labels are unavailable, evaluation is ambiguous, or only a single frozen model is accessible.

Overview We therefore focus on the *self-contained, single-model* setting and study *on-policy self-distillation* as a mechanism for improving reasoning efficiency by letting a model rewrite its own reasoning traces. In our setup, the teacher and the student are the same model. While the student conditions on the problem statement to generate solutions, the teacher conditions on the problem, the student’s generation, and an instruction to rewrite the reasoning trace in a way such that it becomes shorter if the model deems the problem as easy, while retaining or expanding the trace if it deems the problem as hard. Notably, this requires no external correctness or difficulty signals. Thus, we aim to achieve adaptive compute for reasoning, allocating more test-time computation only for problems that really need it.

We operationalize this adaptivity via *self-refinement* as a training signal and on-policy distillation as the mechanism that transfers the signal into the model weights. In an on-policy distillation loop, the same model plays both student and teacher roles. The student generates an on-policy trace, and the teacher is instructed to rewrite and optimize it into a solution with difficulty-adaptive length. Importantly, the rewriting instruction and the additional context induce a conditional distribution over next tokens that is different for the teacher compared to the student. The teacher shifts probability mass toward tokens that appear in the rewritten trace, and away from redundant or low-quality steps. When we score the student tokens under this rewrite-conditioned teacher (i.e., compute teacher log-probabilities on the student-generated next tokens along these prefixes), each position is effectively “graded” by the teacher, yielding a directional, token-level training signal. Because the teacher evaluates the student’s own prefixes, on-policy distillation provides dense, token-level guidance in the states the student actually visits, mitigating the issue of compounding-error in off-policy imitation while avoiding the sparsity of sequence-level RL [11, 12]. In our self-contained setting, the teacher is the same model under a rewrite instruction, hence this dense supervision does not require ground-truth answers or separate reward models [12].

This setup raises concrete questions about (i) whether self-refinement can reliably improve efficiency without compromising correctness, (ii) which prompting and decoding choices yield clean, in-distribution rewrites, (iii) how to evaluate rewrite quality and detect over- and underthinking beyond token counts, and (iv) whether the student can successfully acquire the capability of adaptive compute via token-level on-policy distillation. We therefore ask:

1. Can a model reliably rewrite its own on-policy reasoning traces into traces length-optimized for the respective problem difficulty without external correctness or difficulty signals while preserving robust reasoning?

2. Which rewrite prompting and decoding choices produce rewrites that remain in the distribution of normal solutions, avoiding rewrite-related meta-text and format mismatch?
3. How should redundancy, underthinking, and rewrite quality be defined and measured beyond token counts, with metrics that diagnose failure modes?
4. How does self-rewriting quality vary with model size and decoding settings?
5. Can on-policy rewrite-distillation be used to train a model to generate shorter traces with adequate length for problem difficulty?

The first four questions focus on rewriting and its evaluation. They motivate a benchmark, which evaluates prompt and decoding choices and analyses failure modes, such as answer regressions and meta-text contamination. The fifth question describes the intended training extension via token-level feedback.

Contributions

- We define a self-contained, self-refinement setup for improving reasoning efficiency, where the rewriter conditions only on the problem and the model’s own reasoning trace.
- We introduce a benchmark for rewrite prompt selection and analysis, with diagnostics that go beyond token counts.
- We benchmark rewrite prompts and decoding settings, and document recurring failure modes and prompt design principles needed to keep rewrites compatible with token-level training.
- We specify an on-policy rewrite-distillation loop that uses token-level log-probabilities of a same-model teacher on student-generated prefixes to provide dense supervision.
- We report preliminary on-policy self-distillation results on MATH500 and AIME2025, showing substantial trace-length reductions while suffering from a slight drop in accuracy, particularly at larger model scales.

2. Background

2.1. Problem Setup

We consider tasks where a model receives a prompt x and produces both an answer y and an explicit reasoning trace t . The trace is treated as part of the model output and is expected to be a chain-of-thought style sequence exploring the problem space for different paths to the solution before reaching a final answer. We write the model as a conditional distribution over answers and traces, $p_\theta(y, t \mid x)$, where θ denotes the model’s parameters.

Throughout this thesis, we distinguish a *student* role, which generates traces, from a *teacher* or *rewriter* role, which provides token-level supervision (e.g., soft targets or log-probability scores) for student-generated prefixes. Usually, in distillation tasks, the teacher is a larger, more capable model [13]. However, in our work, both roles are instantiated by the same language model. The teacher role differs only in its prefix prompt in which it receives a problem, the student reasoning trace and instructions to rewrite the reasoning trace.

2.2. Token Probabilities and Distillation Objectives

2.2.1. Token Log-Probabilities Under a Prefix

We model generation over an an auto-regressive policy that predicts one token at a time. Let x denote a fixed prefix (for example, the prompt shown to a model), and let $y = (y_1, \dots, y_T)$ be a sampled completion. The log-probability of the full sequence y under a model π conditioned on x factorises as

$$\log \pi(y \mid x) = \sum_{t=1}^T \log \pi(y_t \mid x, y_{<t}), \quad (2.1)$$

where $y_{<t} = (y_1, \dots, y_{t-1})$ are the previously generated tokens. Informally, “computing the teacher log-probabilities on y ” means evaluating the likelihood of the fixed completion y under the teacher. At each step t , we condition on $(x, y_{<t})$ and record the log-probability assigned to the next observed next token y_t .

2.2.2. Supervised Fine-Tuning

Supervised fine-tuning [14] trains a model on a dataset of reference sequences y^* via teacher forcing, by maximising the likelihood of each reference next token under the reference prefix.

For a dataset $\mathcal{D}$ of input–output pairs (x, y^*) , a standard objective is token-level cross-entropy,

$$\mathcal{L}_{\text{SFT}}(\theta) = -\mathbb{E}_{(x, y^*) \sim \mathcal{D}} \left[\sum_{t=1}^{|y^*|} \log \pi_{\theta}(y_t^* \mid x, y_{<t}^*) \right]. \quad (2.2)$$

In particular, SFT is token-wise in the sense that it matches the probability of a single ground-truth next token at each position, while conditioning on the ground-truth prefix. One key limitation for generation is that at inference time the model conditions on its own sampled prefixes, which can differ from the training prefixes, meaning that the model trains on teacher prefixes, that it might not ever be able reach itself. This is known as *exposure bias* [15].

2.2.3. Knowledge Distillation

Knowledge distillation (KD) replaces hard next-token targets with *soft* teacher targets by not only maximizing the probability of the reference token, but learning the entire next-token distribution under each prefix [13]. A common formulation minimises a token-level divergence between student and teacher next-token distributions along a sequence,

$$\mathcal{L}_{\text{KD}}(\theta) = \mathbb{E}_{x \sim \mathcal{D}} \left[\mathbb{E}_{y \sim q(\cdot \mid x)} \left[\sum_{t=1}^{|y|} D(\pi_{\theta}(\cdot \mid x, y_{<t}) \parallel \pi_{\text{teach}}(\cdot \mid x, y_{<t})) \right] \right], \quad (2.3)$$

where $q(\cdot \mid x)$ is the distribution over sequences that determines which prefixes are visited during training. When D is a KL divergence, swapping its direction yields different behavior: $\text{KL}(\pi_{\text{teach}} \parallel \pi_{\theta})$ is mode-covering and can be implemented as token-level cross-entropy on teacher soft targets, whereas $\text{KL}(\pi_{\theta} \parallel \pi_{\text{teach}})$ is mode-seeking and is convenient in on-policy settings because it can be estimated by sampling from the student and scoring those tokens under the teacher.

Distillation is therefore fundamentally token-level for auto-regressive models as even a KL defined over full sequences factorises into a sum of per-token terms under the auto-regressive decomposition. The phrase *sequence-level distillation* is commonly used to refer to how targets are obtained, for example, distilling from teacher-generated sequences rather than a fixed dataset, while still optimising a token-level likelihood objective on those sequences [16].

2.2.4. Self-Distillation and Context Distillation

In self-distillation, the teacher and student share the same base architecture as well as, in our case, the same initial weights, and the teacher role is realised through a different checkpoint, ensembling, or a different conditioning strategy [17, 18]. We use a same-model setting where teacher and student roles are instantiated by the same underlying language model, differing only in their context.

In context distillation, the teacher is allowed to condition on additional context c that is unavailable to the student at inference time, and the student is trained to approximate the teacher behavior without access to c [19]. In our setting, this asymmetry arises because the teacher receives extracted context derived from the student’s own thinking trace. This

perspective also aligns with context distillation, which combines a context-conditioned teacher with on-policy data collection [20]. Concretely, a context-conditioned teacher has next-token distribution $\pi_{\text{teach}}(\cdot \mid x, c, y_{<t})$, while the student has $\pi_{\theta}(\cdot \mid x, y_{<t})$ at inference time. Distilling such a teacher into a context-free student requires both (i) choosing which prefixes $(x, y_{<t})$ to train on and (ii) defining an objective that matches next-token distributions along those prefixes. In our case, the additional context c is derived from the student and is used to guide rewriting.

2.3. On-Policy Distillation

Standard distillation for sequence models can suffer from train-inference mismatch [15, 21], because the student is trained on sequences that come from a fixed dataset or from a teacher [16, 13], but at deployment time it conditions on its own generated prefixes. On-policy distillation addresses this by sampling sequences from the current student and using the teacher to provide token-level feedback on those exact student-generated prefixes [11, 12]. Concretely, given an input prompt x , one samples a student completion $y \sim \pi_{\theta}(\cdot \mid x)$, and then trains the student so that its next-token distributions are close to the teacher’s along the visited states $(x, y_{<t})$.

A convenient view is to define a token-level divergence $D(\cdot \parallel \cdot)$ between next-token distributions and minimise an expected on-policy objective of the form

$$\mathcal{L}(\theta) = \mathbb{E}_{x \sim \mathcal{D}} \left[\mathbb{E}_{y \sim \pi_{\theta}(\cdot \mid x)} \left[\sum_{t=1}^{|y|} D(\pi_{\theta}(\cdot \mid x, y_{<t}) \parallel \pi_{\text{teach}}(\cdot \mid x, y_{<t})) \right] \right], \quad (2.4)$$

which aligns the student with the teacher on the distribution of prefixes that the student itself induces [11]. Different choices of D correspond to different distillation behaviours and trade-offs. For example, the *forward* KL $\text{KL}(\pi_{\text{teach}} \parallel \pi_{\theta})$ tends to be mode-covering, whereas the reverse KL $\text{KL}(\pi_{\theta} \parallel \pi_{\text{teach}})$ tends to be mode-seeking [13]. Intuitively, mode covering penalizes the student for assigning low probability to any high-probability teacher continuation, whereas mode seeking encourages concentrating mass on a subset of teacher continuations. This matters for rewriting because a teacher may admit multiple valid rewritten traces and a reverse-KL objective encourages the student to commit to one of them (e.g., a consistent concise rewrite style) rather than spreading probability mass across many alternatives. We use a reverse-KL style distillation objective.

Reinforcement learning (RL) provides a general framework for optimising sequence-level objectives via scalar rewards, but for language modeling this often implies sparse feedback, difficult credit assignment, and high-variance updates [22, 14]. In contrast, on-policy distillation provides dense supervision at every token position and directly targets the student’s next-token distribution on the prefixes the student actually visits. In our setting, this is especially useful because the teacher signal is obtained without ground-truth answers or separate reward models. The teacher is the same model under a different prompt, which yields token-level guidance by shifting probability mass away from redundant reasoning and toward more efficient traces. RL remains valuable when an explicit task reward is available

and the desired behavior cannot be expressed as matching a teacher distribution, but in the self-contained rewriting setting the stability and density of token-level distillation are an advantage.

2.4. Reasoning Traces and Rewriting

2.4.1. Redundancy and Compression

To study rewriting, a definition of redundancy is needed. One perspective comes from work on rationales, where an explanation is desirable if it is sufficient, meaning removing parts of the input or rationale should not change the model’s prediction [23]. Translated to reasoning traces, a trace can be considered redundant if large segments can be removed, or rewritten more compactly, without changing the answer that the reasoning eventually leads to. Previous work has shown that training on reasoning traces where redundant segments are removed can decrease model token spending while preserving accuracy [9].

Another perspective is based on token complexity. Compression studies argue that there is an instance-dependent minimum number of tokens that are needed to preserve accuracy, and that pushing below this level leads to failures even if average compression looks good [5]. This view suggests that a good rewriting method should be adaptive meaning that easy instances can be compressed more aggressively, while harder instances require longer traces to preserve correctness.

There are also risks in rewriting traces. For tasks that benefit from exploration and backtracking, such as search-like reasoning, removing intermediate steps may eliminate useful verification or alternative branches [24]. In addition, reasoning traces are not guaranteed to be faithful explanations of a model’s internal computation, which complicates interpretation. A shorter trace is not automatically more truthful, and rewriting may change the narrative without improving the underlying decision process [25, 26]. For this reason, we treat rewritten traces primarily as a tool for efficiency and utility, while recognizing that faithfulness is a separate evaluation dimension.

2.4.2. Distribution and Format Mismatch

Token-level distillation assumes that the teacher probabilities are informative for the kind of outputs we want the student to produce. If the teacher is prompted in a way that shifts its output distribution to a different format or schema, for example, producing rewrite-related meta-text, edit instructions, or a different answer format, then scoring student-generated solution tokens under this shifted teacher can become uninformative or systematically biased. In terms of the distillation objective, the teacher distribution $\pi_{\text{teach}}(\cdot \mid x, y_{<t})$ no longer represents a “better solution policy” on the student-visited prefixes, so minimizing cross-entropy or KL can push the student toward matching the wrong distribution. This motivates careful prompt design to keep teacher outputs and token probabilities close to the distribution of the student.

2.4.3. Rewrite Benchmarking

For the reasons above, we treat prompt selection for rewriting as a prerequisite and create a benchmark to compare rewrite prompts and decoding settings before introducing on-policy training. The benchmark evaluates rewrites at the level of individual instances by comparing an original trace and a rewritten trace along multiple axes: (i) final-answer correctness, (ii) token reduction and token-level efficiency, (iii) step-level completeness and correctness (recall and precision against reference key steps), and (iv) reflection quality, which measures whether self-corrections in the trace are productive or redundant. The full pipeline and metric definitions are described in Section 4.1.1 and Section 4.1.2.

3. Related Work

3.1. Reasoning Models and RL-Incentivized Long-Form Reasoning

3.1.1. RL post-training teaches models to “think longer”

Recent reasoning models are explicitly trained to produce useful intermediate reasoning, and RL post-training plays a central role in shaping this behavior. OpenAI o1 [3] was trained with a large-scale RL, improving reasoning quality and yielding *test-time compute scaling*: Performance improving when the model is allowed to spend more compute at inference time. DeepSeek-R1 [2] similarly uses reinforcement learning to incentivize reasoning behaviours, showing that RL training can yield emergent, long chains-of-thought and self-verification patterns. A training paradigm that incentivizes ever longer reasoning traces motivates methods that are able to re-compress intelligence in order to maximize intelligence-per-dollar.

3.1.2. Overthinking and Wasted Compute in Reasoning

RL-incentivized reasoning does not imply that more reasoning is always beneficial. Empirical analyses of reasoning models report overthinking, which means allocating substantial computation to simple problems with little accuracy gain [4]. More broadly, recent work [27, 28] finds that the relationship between reasoning length and correctness can be problem-dependent and non-monotonic. Cross-model comparisons also suggest that improved reasoning may reflect a different allocation of computation rather than simply longer outputs [29]. Difficulty-adaptive slow-thinking frameworks aim to reduce wasted tokens on easy instances while preserving structure for hard instances [30], and adaptive compression methods explicitly target under-/overthinking trade-offs [31].

This motivates our problem framing: Rather than enforcing a global token budget, we study rewriting as a problem-adaptive mechanism to reduce redundant tokens while avoiding underthinking on hard problems.

3.2. Test-Time Compute Scaling

3.2.1. Prompting, sampling, search, and refinement-style reasoning

Beyond RL post-training, many approaches improve reasoning by spending additional test-time compute. Classic prompting methods elicit intermediate reasoning steps via chain-of-thought (CoT) prompting [1]. Sampling-based methods scale compute by drawing multiple candidates and selecting or aggregating them, including self-consistency voting [32] and

best-of- N selection with a verifier or reward model [33]. Search-style methods treat intermediate reasoning as a search problem, ranging from Tree-of-Thought style explicit state search [24] to verifier-guided decoding and tree search against process-based verifiers [33]. A complementary line of work improves outputs through iterative critique-and-revise or self-refinement loops that sequentially update the proposal distribution at test time [34, 33]. Other compute-scaling mechanisms include multi-agent debate [35] and tool/program-aided reasoning, where the model offloads parts of the computation to an external executor [36].

Many of these methods increase compute primarily via branching and selection across multiple candidates, explicit search guided by verifiers, or sequential refinement through revisions. In contrast, we investigate methods that simply autoregressively sample from the model distribution at inference time, without any branching, verifier or reward model. We study whether self-refinement can remove redundancy within a single on-policy trace while keeping outputs in-distribution for downstream distillation.

3.2.2. Adaptive Inference Policies and Early Stopping

Another direction allocates test-time compute adaptively rather than uniformly across inputs. Input-adaptive compute allocation methods learn policies that predict when additional decoding computation (e.g., more samples or a more expensive procedure) is likely to improve output quality and route inputs accordingly [37, 38, 39]. Related work targets the cost of self-consistency by proposing early stopping rules or difficulty-adaptive sampling strategies to reduce the number of samples needed for multi-step reasoning [40, 41]. Difficulty estimation can also be used to adjust prompting or reasoning style at inference time [42].

Many adaptive compute approaches rely on auxiliary predictors, routing policies, or explicit difficulty signals. Our approach is self-contained, meaning the rewriter conditions only on the problem and the model’s own trace.

3.3. Compression, Pruning, and Length Control

3.3.1. Prompt- and decoding-level length control

Several approaches reduce reasoning cost by constraining output length or making intermediate steps more concise. Token-budget-aware reasoning conditions generation on a prescribed budget [6], while Chain-of-Draft aims to keep multi-step structure but compress each step to fewer tokens [7]. Systematic studies of compression emphasize that accuracy-length trade-offs are instance-dependent and that pushing compression below an instance-specific threshold can harm correctness [5, 9].

This motivates our emphasis on instance-adaptive rewriting: rather than choosing a single global budget, rewriting can in principle compress aggressively on easy instances and preserve or expand reasoning where needed.

3.3.2. Training-Time Approaches for Conciseness

Beyond prompting, training-time methods aim to internalize concise reasoning behavior. Self-training can elicit shorter reasoning traces by using model-generated data and selection criteria that favor conciseness [43]. Fine-tuning approaches such as pruning target long-trace reasoning models by harmonizing or shortening traces during post-training [44]. Reinforcement learning can also expose a “length control” knob or regularize reasoning length for specific domains, for example by training policies that trade off correctness and token usage [8, 45, 46].

3.4. Evaluating Reasoning Traces and Thinking Efficiency

Evaluating reasoning traces requires criteria beyond final-answer correctness and raw token counts. A survey of step-by-step trace evaluation proposes dimensions such as logical validity, coherence, and usefulness for reaching the answer [47]. Dedicated benchmarks such as THINK-Bench focus on thinking efficiency and chain-of-thought quality for reasoning models, highlighting that models can be accurate yet inefficient depending on how they allocate test-time reasoning [48].

These perspectives directly motivate the design of a benchmark as described in Section 4.1.1.

3.5. Self-Improvement, Rewriting, and Distillation

Rewriting and feedback are widely used mechanisms for self-improvement. Self-Refine [34] proposes iterative critique-and-edit loops to improve outputs, and STaR [49] bootstraps reasoning by generating rationales that can be re-used as supervision. Distillation provides a general mechanism to transfer improved behaviors into a student model [13], and on-policy variants mitigate distribution shift by aligning the student on prefixes it actually visits [11, 12]. Context distillation frames settings where a teacher conditions on additional context unavailable at deployment; formal definitions and how this perspective applies to our trace-conditioned teacher are given in Chapter 2 [50, 19, 20].

Recent training pipelines for modern agentic and reasoning models also employ rewriting as a *data synthesis* mechanism. The Kimi K2 technical report describes an autoregressive long-document rewriting pipeline that rewrites text chunk-by-chunk while conditioning on previously rewritten chunks, and applies fidelity verification to ensure the rewritten chunks preserve the meaning of the original document [51]. In particular, their *Mathematics Data Rephrasing* procedure rewrites high-quality mathematical documents into a learning-note style to create additional training data while preserving mathematical content [51].

4. Method

This thesis follows a two-stage method. First, we benchmark rewrite prompts and decoding settings to establish that self-refinement is feasible and to select a prompt that produces solution outputs without rewrite-related meta-text. Second, we specify an on-policy self-distillation loop that distills the latent rewriting capability into the model’s default distribution via token-level feedback.

4.1. Rewriting Benchmark

4.1.1. Pipeline and Artifacts

Our proposed benchmark is an instance-level benchmark used to select rewrite prompts and decoding settings and evaluates rewriting beyond final answers and token counts.

For each problem instance x , we generate an original completion y_{orig} and a rewritten completion y_{rew} and compute a set of metrics, which evaluates the generation quality. In our experiments, we restrict x to mathematics problems and only evaluate the thinking trace between `<think>` and `</think>` of y .

1. **Student generation:** The student role is prompted with x and samples an answer that contains a reasoning trace and a final answer, denoted y_{orig} . We refer to Section 5.1 where we instantiate different model sizes from the Qwen-3 [52] series as students.
2. **Self-refinement:** The same model is instantiated as a rewriter with an explicit rewrite instruction prompt. It is conditioned on $(x, c(y_{\text{orig}}))$, where $c(\cdot)$ extracts the student thinking trace between `<think>` and `</think>`, and it outputs an answer y_{rew} without rewrite-related meta-text.
3. **Reference key step generation:** A more capable judge model generates up to three distinct reference approaches for x and extracts ordered key steps for each approach given the ground truth solution of x . For our experiments we use DeepSeek V3.2 [53] via the Microsoft Azure Foundry API¹.
4. **Scoring and aggregation:** We compute outcome and token metrics from the extracted thinking traces of y_{orig} and y_{rew} , as well as the judge-based metrics for efficiency, step-level recall and precision, and reflection quality, and report the difference in scores for the original and rewritten thinking traces.

¹<https://azure.microsoft.com/en-us/products/ai-foundry>

4.1.2. Metrics and Extraction

The metrics, methods and prompts used to compute them are based on prior work on benchmarking reasoning traces in LLMs, such as THINK-Bench [48] and MME-CoT [54].

Correctness and answer extraction. Each completion y is mapped to an extracted final answer $\hat{a}$ using boxed answers (content inside `\boxed{...}`). Correctness is then computed by an equivalence check `math_equal( $\hat{a}, a^*$ )` where a^* is the ground truth answer that combines tolerant numerical comparison with symbolic equivalence when needed.

Token Counts and Compression. We measure token counts using the source model tokenizer. When completions contain explicit thinking segments, we report trace token counts separately. We define the compression ratio as $T_{\text{trace}}(y_{\text{rew}})/T_{\text{trace}}(y_{\text{orig}})$ and the absolute token reduction as $T_{\text{trace}}(y_{\text{orig}}) - T_{\text{trace}}(y_{\text{rew}})$.

Efficiency. To measure structural efficiency, we estimate where in the trace the model first commits to the final answer it submits. The judge identifies the earliest sentence or span that commits to the submitted answer. We locate this span in the trace and count tokens from the start of the trace to the end of the span. The efficiency score is the ratio of this prefix length to the total trace token count. If the span cannot be located, the score defaults to 1.0. This metric expresses how much an LLM tends to explore different reasoning paths and correct itself versus how much it commits to an answer early, without overchecking.

Step-Level Evaluation. Step-level evaluation uses evaluation-only reference steps. The judge model generates up to three distinct reference approaches for x and extracts key steps for each. For a candidate trace, the judge decomposes the text into a sequence of atomic reasoning steps and assigns each a type in `{logical_inference, background_info, other}`. The judge then selects the best-matching approach and evaluates recall by matching reference key steps. We compute recall as `matched/total reference steps`. For precision, the judge labels each `logical_inference` step as `Match`, `Reasonable`, or `Wrong`. We report strict precision as `Match/N` and soft precision as `(Match + Reasonable)/N`, where N is the number of evaluated `logical_inference` steps.

Reflection Quality. To evaluate self-corrections, the judge identifies reflection segments in the trace and labels each reflection as correct or wrong relative to the reference. We report the reflection score as `correct reflections/total reflections`, and we also report the total number of reflections. If no reflections are identified, the reflection score defaults to 1.0.

4.1.3. Failure Modes and Diagnostics

Several recurring failure modes can occur when rewriting reasoning traces. This section defines some failure modes and links each to the metrics and detectors used in the benchmark. Concrete examples and their frequency in the benchmark are reported in Chapter 5.

1. **Trace dropout (skips thinking):** The teacher outputs only a final answer or a very short trace that omits essential reasoning steps. This failure is diagnosed by a collapse in trace token counts and step-level recall, and it is often correlated with format non-compliance if a trace schema is required.
2. **Parroting (copying without improvement):** The teacher repeats the student trace with minimal edits and achieves little token reduction. This failure is visible in compression statistics and in high textual overlap between original and rewritten traces.
3. **Meta-text contamination (distribution mismatch):** The teacher produces rewrite-related meta-text (e.g., announcing the rewrite) instead of a normal solution completion. This is diagnosed via format-compliance checks and meta-text contamination detectors, and it can also be visible as a mismatch in teacher log-probabilities under token-level distillation.
4. **Answer drift:** The rewritten trace changes intermediate steps in a way that changes the extracted final answer. This is diagnosed by answer extraction and answer-flip statistics (regressions versus corrections).
5. **Over-compression:** The teacher deletes steps that are required for harder instances, producing traces that are too short to support the correct answer. This typically manifests as token reduction combined with drops in correctness and step-level recall.

4.2. On-Policy Rewrite and Distillation Loop

The loop alternates between sampling *live* on-policy rollouts from the current student and updating the student using token-level supervision from a rewrite-conditioned teacher distribution. Importantly, the teacher does not explicitly rewrite the live rollout. Instead, it conditions on an *offline, static* student trace for the same instance and assigns log-probabilities to the live student tokens under this rewrite-conditioned context (implicit rewriting). The student trace, used for conditioning the teacher, is collected before any training. The teacher remains fixed at the initial weights throughout training, while the student is updated.

4.2.1. Concrete Prompt and Context Construction

This section fixes notation for how the student trace is turned into teacher context. In our experiments, x is a mathematics problem instance, and traces are the text between `<think>` and `</think>` (i.e., $c(\cdot)$ extracts only the reasoning trace, not the final answer text). We write the student prompt template as

$$p_s(x) = \text{"Please reason step by step, and put your final answer within \boxed{ }. " || } x, \quad (4.1)$$

where $\|$ denotes string concatenation. Autoregressively sampling the student model under this prefix yields a completion token sequence $y = (y_1, \dots, y_T) \sim \pi_\theta(\cdot \mid p_s(x))$, where $T = |y|$

denotes its length. In our setting, y contains an explicit trace segment, for example:

$$y = "<\text{think}> \dots \text{reasoning tokens} \dots </\text{think}> \text{Final answer: } \dots". \quad (4.2)$$

We define a trace extractor $c(\cdot)$ that maps a completion to a trace string,

$$c(y) = t, \quad (4.3)$$

where t is the substring inside the `<think>` tags.

Static trace context. In our training setup, the teacher does not condition on the reasoning trace extracted from the live on-policy rollout. Instead, for each problem x in a fixed training dataset, we first generate a *static* student completion $\tilde{y}(x)$ once using an initial (not yet distilled) student checkpoint and extract its trace

$$t_{\text{static}}(x) = c(\tilde{y}(x)). \quad (4.4)$$

Toy example (static trace vs. live rollout). For illustration, consider a math instance

$$x = \text{"Solve for n: } 1+2+\dots+n = 120\text{."}. \quad (4.5)$$

We generate one baseline completion $\tilde{y}(x)$ offline and store only its extracted trace $t_{\text{static}}(x) = c(\tilde{y}(x))$. During training, the current student separately samples a *live* completion y for the same x . These two traces do not need to match, the key point is that $t_{\text{static}}(x)$ is a fixed per-instance context, while y changes as the student is updated.

Teacher prefix with rewrite instruction. The teacher is conditioned on the original problem and the static trace via a teacher prefix template that includes an explicit rewrite instruction. The instruction is composed of:

$$p_t(x, t_{\text{static}}(x)) = [\text{Rewrite Instruction}] \parallel "[\text{Problem}] \parallel x \parallel "[\text{Student Trace (static)}] \parallel t_{\text{static}}(x). \quad (4.6)$$

This prefix defines a rewrite-conditioned teacher distribution: if sampled, it would produce a normal solution completion whose trace is a rewritten version of the provided static student trace. In training, the teacher is used only as a scorer (single forward-pass per trace) on the live student tokens and is not sampled to produce explicit rewrites.

4.2.2. Teacher Log-Probabilities on Student Tokens

The teacher distribution is conditioned on the teacher prefix $p_t(x, t_{\text{static}}(x))$ (which includes the rewrite instruction and the static student trace), while the scored sequence is the exact live student-generated token sequence $y = (y_1, \dots, y_T)$. Concretely,

$$\log \pi_{\text{teach}}(y \mid p_t(x, t_{\text{static}}(x))) = \sum_{\tau=1}^T \log \pi_{\text{teach}}(y_{\tau} \mid p_t(x, t_{\text{static}}(x)), y_{<\tau}). \quad (4.7)$$

At each position τ , the teacher is evaluated on the prefix $(p_t(x, t_{\text{static}}(x)), y_{<\tau})$, where $y_{<\tau}$ are the past tokens sampled by the student, and we read off the probability assigned to the student’s next token y_τ . All student tokens are scored in parallel in a single teacher forward-pass, thanks to the causal mask in LLMs [55].

Thus our method fits the framing of on-policy self-distillation or on-policy context distillation [11, 20], where trajectories are sampled from the student and the teacher provides token-level supervision, while the teacher is the same model as the student, additionally conditioned on extra context $t_{\text{static}}(x)$ that the student does not observe at inference time.

4.2.3. Training Loop

With the prompt and scoring setup fixed above, we can now describe the on-policy rewrite-distillation loop as a sequence of concrete steps.

Step A: Precompute static traces (offline). For each training instance x in a fixed dataset, we generate one static student completion $\tilde{y}(x)$ using an initial student checkpoint and store the extracted trace $t_{\text{static}}(x) = c(\tilde{y}(x))$. This step is performed once to create a fixed trace context per instance.

Step B: Generate live on-policy rollouts. During training, for each instance x , the current student is prompted with $p_s(x)$ and samples a live completion $y \sim \pi_\theta(\cdot \mid p_s(x))$. This rollout is on-policy by construction.

Step C: Implicit rewriting via teacher scoring. The teacher is instantiated from the initial model weights and run under the rewrite-conditioned prefix $p_t(x, t_{\text{static}}(x))$. We compute teacher token log-probabilities on the *live* student rollout y under this prefix, as described in Section 4.2.2. Using the rewrite instruction and static trace, we aim to shift the teacher’s conditional distribution toward a more compute optimal strategy, nudging the model away from redundancy and overthinking. We refer to this as *implicit rewriting*. The scored tokens are the student’s live completion tokens.

Step D: Update the student by on-policy distillation. Using the teacher scores on the student rollouts, we update the student parameters to reduce the divergence between student and teacher next-token distributions. By scoring the student generations and thus staying its on-policy, the student exclusively receives training signal for the prefixes it actually sees [11]. Because the teacher conditions on $t_{\text{static}}(x)$ while the student does not, this update can be viewed as distilling a context-conditioned teacher into a context-free student [20]. We use the reverse KL divergence $D = D_{\text{KL}}(\pi_\theta \parallel \pi_{\text{teach}})$.

5. Experiments

5.1. Experimental Setup

This chapter reports the benchmark results as well as results from preliminary training. We first specify datasets, models, decoding, and prompts. Exact prompt templates for generation, rewriting and benchmark metrics are listed in Appendix A.

5.1.1. Datasets and sampling

Our benchmark is instantiated on OpenThoughts-114k [56] and restricted to the mathematics problems of the dataset. We sample a fixed subset of 1500 problems with a deterministic seed 42 to ensure identical instance sets across models. We use the recommended prompt template for math problems [57]:

Please reason step by step, and put your final answer within `\boxed{}`.

5.1.2. Used models and decoding settings

The benchmark uses the same model weights in both the student and rewriter roles, instantiated via different prompts and trace context. We use models from the Qwen3 series [52] and evaluate Qwen3-{1.7B, 4B, 8B, 14B}. On-policy distillation results are reported for the same set of model sizes and are conducted using the same decoding settings. All evaluation-only judge components are implemented with We use DeepSeek-V3.2 [53] as our judge throughout our experiments..

Unless otherwise stated, for the student and rewriter we use temperature 0.6, top- p 0.95, top- k 20, min- p 0, and a maximum of 16,384 output tokens, with non-greedy decoding, as recommended by the model providers [57]. For the judge, we also use the default decoding parameters, with a temperature of 1.0, top- p 1.0, presence penalty 0, and frequency penalty 0 [53].

5.2. Benchmark Results

We report the benchmark metrics defined in Chapter 4, Section 4.1.2. For readability, we group them into four categories: outcome metrics measure final-answer correctness; efficiency metrics measure token counts, compression ratio, and the efficiency score; step-level quality metrics measure recall and precision against reference solution key steps (strict and soft);

Table 5.1.: Benchmark results summary on the mathematics subset (1500 instances per model). Metrics are aggregated over instances and reported separately for original and rewritten traces.

Model	Trace	Accuracy (%)	Average tokens per generation	Token reduction (%)	Efficiency score	Step recall	Step precision	Reflection score
Qwen3-1.7B	<i>Original</i>	53.3	7638	–	0.651	0.841	0.459	0.667
	<i>Rewritten</i>	57.3	1405	81.6	0.802	0.764	0.490	0.668
Qwen3-4B	<i>Original</i>	58.0	7355	–	0.600	0.890	0.517	0.765
	<i>Rewritten</i>	63.3	1250	83.0	0.799	0.857	0.542	0.777
Qwen3-8B	<i>Original</i>	60.0	7579	–	0.614	0.901	0.508	0.814
	<i>Rewritten</i>	66.0	1421	81.3	0.796	0.864	0.484	0.797
Qwen3-14B	<i>Original</i>	65.3	6233	–	0.611	0.928	0.517	0.818
	<i>Rewritten</i>	66.3	1340	78.5	0.792	0.870	0.553	0.806

and reflection quality captures both the number of reflections and a reflection score. The benchmark code is available in our public repository; see Appendix A.1.

5.2.1. Discussion

Self-Rewriting Feasibility. Table 5.1 shows that same-model rewriting yields substantially shorter traces while preserving—and often improving—final-answer correctness. Across model sizes, average tokens drop from roughly 6–8k to roughly 1.2–1.4k (78–83% reduction) while accuracy increases by +1.0 to +6.0 points. Step recall remains high (0.84–0.93 → 0.76–0.87), but the consistent recall drop indicates occasional omission of reference-critical steps, i.e., an underthinking risk under heavy compression. Validating instance-level adaptivity would require stratifying token reduction and accuracy by difficulty.

Prompting and Decoding for in-distribution Rewrites. We can compute the full metric suite for rewritten traces across all four model sizes, and rewritten traces show consistent efficiency and accuracy gains (Table 5.1). This indicates that the rewrite prompt template (Appendix A) and non-greedy decoding (Section 5.1.2) keep rewrites close to the *normal-solution* schema needed for parsing and judge-based evaluation, and therefore suitable as a teacher distribution for token-level training. In contrast, rewrite prompts that induce meta-text or format mismatch would surface as answer-extraction failures, reduced benchmark coverage, and unstable teacher scoring (Section 4.1.3).

Measuring Redundancy and Underthinking beyond Token Counts. Token counts alone miss important trade-offs. While compression is extreme, the step metrics separate productive compression from over-compression: recall drops modestly while strict precision stays similar or increases (e.g., Qwen3-4B: 0.517 → 0.542; Qwen3-14B: 0.517 → 0.553), consistent with pruning redundant or incorrect content while largely preserving core structure. Rewrites also increase the efficiency score (about +0.18 to +0.20), indicating earlier commitment with less backtracking, while reflection scores remain broadly stable (Table 5.1).

5.3. On-Policy Self-Distillation Training and Results

5.3.1. Training

We train the student using the on-policy rewrite-distillation loop described in Section 4.2. Concretely, the student samples *live* solutions on a fixed training set, while a same-model teacher (kept fixed at the initial weights) provides token-level supervision under a rewrite-conditioned prefix that includes a per-instance static student trace (precomputed once before training). The on-policy self-distillation training code used for these experiments is available in our public repository; see Appendix A.1.

5.3.2. Training Configuration

We train the student using the on-policy rewrite-distillation loop described in Section 4.2. Concretely, the student samples *live* solutions on a fixed training set, while a same-model teacher (kept fixed at the initial weights) provides token-level supervision under a rewrite-conditioned prefix that includes a per-instance *static* student trace (precomputed once before training).

All models are trained with AdamW [58] using a learning rate of 1×10^{-5} , weight decay 0.01, $\beta = (0.9, 0.999)$, and a linear warm-up over 10 steps followed by a constant schedule. We train all model sizes over 100 steps in total. The training objective is a reverse KL divergence $D_{\text{KL}}(\pi_\theta \parallel \pi_{\text{teach}})$ computed over the top- $k=512$ teacher logits per position, with all logits outside the top- k set to zero before the softmax, which reduces memory and focuses the distillation signal on the most probable teacher continuations. We use a global batch size of 64 (32 prompts per step, one generation per prompt, micro-batch size 2 with gradient accumulation), gradient clipping at 1.0, and activation checkpointing. The maximum sequence length is 20,480 tokens.

The training set is drawn from the mathematics subset of OpenThoughts-114k [56] with a held-out validation split of 128 instances (seed 42). The student generates on-policy rollouts at temperature 0.6 with a maximum of 16,384 new tokens, matching the benchmark decoding settings (Section 5.1.2). Validation is run every 10 steps, and checkpoints are saved every 10 steps. For each model-size, we report the checkpoint with the smallest accuracy gap relative to the baseline on the downstream evaluations in Tables 5.2 and 5.3.

Training Dynamics

Figure 5.1 shows training loss, validation accuracy, and average generation length across all four model sizes over 100 training steps. Training loss is visualized using a causal exponential moving average (EMA) with smoothing factor $\alpha = 0.08$, computed as $s_t = \alpha x_t + (1 - \alpha) s_{t-1}$ with $s_0 = x_0$, to suppress high-frequency batch noise while preserving the trend. Validation metrics are evaluated on a held-out set of 128 instances from the OpenThoughts dataset at every 10 steps.

All models (Figure 5.1, left) show stable convergence of the on-policy distillation loop. Larger models show a clearer decrease in loss than the smaller model sizes. We hypothesize,

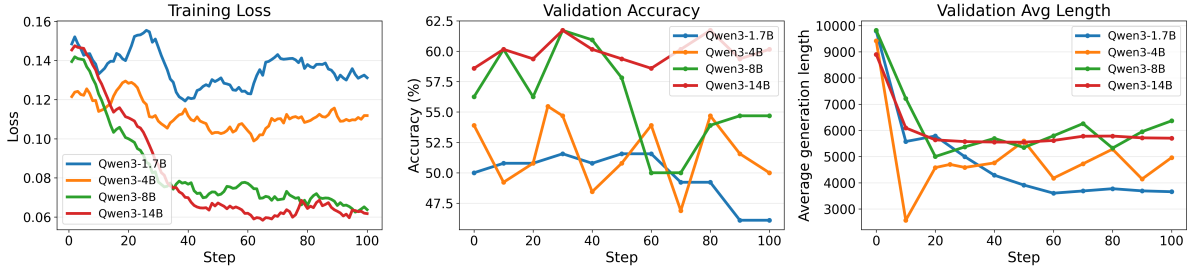


Figure 5.1.: Training dynamics for on-policy rewrite-distillation across Qwen3 model sizes.

Left: Training loss (EMA-smoothed, $\alpha=0.08$) **Center:** Validation accuracy on a held-out set of 128 OpenThoughts instances. **Right:** Average generation length on the validation set.

that larger models are better at rewriting their own traces, thus receiving better learning signal. Notably, the 1.7B converges steadily to a similar value as its initial loss, while the 14B model begins at a similar loss but shows rapid decrease.

Validation accuracy (Figure 5.1, center) exhibits considerable variance due to the small validation set (128 instances), but broad trends are visible: the 14B model maintains the highest accuracy throughout training ($\sim 60\%$), while smaller models fluctuate around 48–55%. The high variance is expected given the small evaluation set and non-greedy decoding.

Average generation length (Figure 5.1, right) shows a clear shortening effect for the 1.7B model, whose average length drops from approximately 9,000 tokens to around 3,500 tokens over the course of training, the strongest compression among all sizes. The 4B and 8B models show moderate length reduction, while the 14B model drops from 9,000 and then remains relatively stable around 6,000 tokens, suggesting that larger models are less aggressive in their token compression when trained using our method. This pattern is consistent with the downstream evaluation, where the 14B model shows the smallest token reduction (-34.5% on MATH500) but also the smallest accuracy drop (-0.9 points).

5.3.3. Downstream Evaluation

The on-policy distillation results show a clear length transfer effect but an incomplete quality transfer effect. Across model sizes, distillation reduces average trace length by roughly 35–46% on MATH500 and 28–44% on AIME2025 (Tables 5.2 and 5.3). However, success rates decrease for all models, with the drop being small on the easier benchmark MATH500 (from -0.9 to -5.6 points) and substantially larger on the harder AIME2025 (from -8.3 to -13.5 points). This indicates that, in the current training setup, the distilled student learns a shorter default reasoning style but does not yet reliably preserve the additional computation needed for hard instances, consistent with the underthinking failure mode in Section 4.1.3. Improving this trade-off likely requires training mechanisms that make the rewrite-conditioned teacher signal more instance-adaptive (so the student stays concise on easy instances while retaining verification on hard ones), rather than uniformly shifting the entire policy toward brevity.

5. Experiments

Table 5.2.: On-policy distillation results on MATH500. We report success rate (Avg@4) and trace tokens per generation for the baseline and distilled checkpoints. Changes are computed as distilled minus baseline. Token limit for generations was 16384 tokens.

Model	Checkpoint	Success rate (Avg@4) (%)	Trace tokens per generation	Change in success rate (percentage points)	Change in trace tokens	Change in trace tokens (%)
Qwen3-1.7B	Baseline	90.9%	5467	—	—	—
	Distilled	85.3%	3087	-5.6	-2380	-43.5%
Qwen3-4B	Baseline	95.5%	5048	—	—	—
	Distilled	92.8%	2717	-2.7	-2331	-46.2%
Qwen3-8B	Baseline	96.1%	5297	—	—	—
	Distilled	94.2%	2881	-1.9	-2416	-45.6%
Qwen3-14B	Baseline	96.2%	4732	—	—	—
	Distilled	95.3%	3099	-0.9	-1633	-34.5%

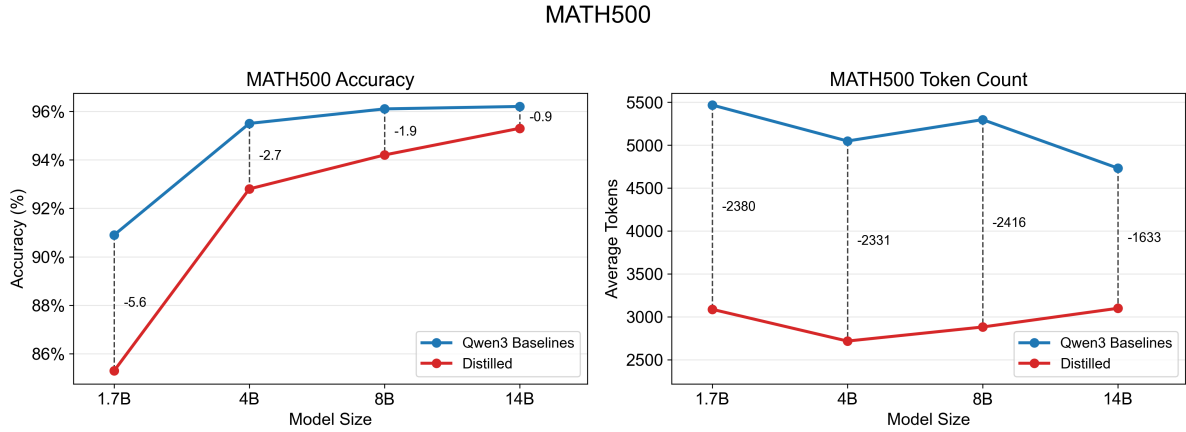


Figure 5.2.: Math500 success rate (Avg@4) and average trace tokens per generation for baseline versus distilled checkpoints across model sizes.

Furthermore, running the training loop on a dataset of harder or more diverse problems might mitigate this issue.

Distillation benefits from Model Scale. The success-rate gap between baseline and distilled checkpoints narrows with model size (Tables 5.2 and 5.3). On MATH500, distillation reduces success by -5.6 points at 1.7B but only -0.9 points at 14B, while still reducing trace tokens by 34–46%. On AIME2025, the drop shrinks from -13.5 points (1.7B) to -8.3 points (14B) under substantial compression (28–44% token reduction). This trend suggests that as the underlying model becomes stronger, the rewrite-conditioned teacher distribution provides more useful token-level guidance, allowing the student to internalize conciseness while retaining more of the robustness required for hard instances.

5. Experiments

Table 5.3.: On-policy distillation results on AIME2025. We report success rate (Avg@4) and trace tokens per generation for the baseline and distilled checkpoints. Changes are computed as distilled minus baseline. Token limit for generations was 16384 tokens.

Model	Checkpoint	Success rate (Avg@4) (%)	Trace tokens per generation	Change in success rate (percentage points)	Change in trace tokens	Change in trace tokens (%)
Qwen3-1.7B	Baseline	35.8%	13193	–	–	–
	Distilled	22.3%	7449	-13.5	-5744	-43.5%
Qwen3-4B	Baseline	52.4%	13061	–	–	–
	Distilled	40.4%	8859	-12.0	-4202	-32.2%
Qwen3-8B	Baseline	54.2%	13328	–	–	–
	Distilled	43.7%	9088	-10.5	-4240	-31.8%
Qwen3-14B	Baseline	60.0%	12572	–	–	–
	Distilled	51.7%	8996	-8.3	-3576	-28.4%

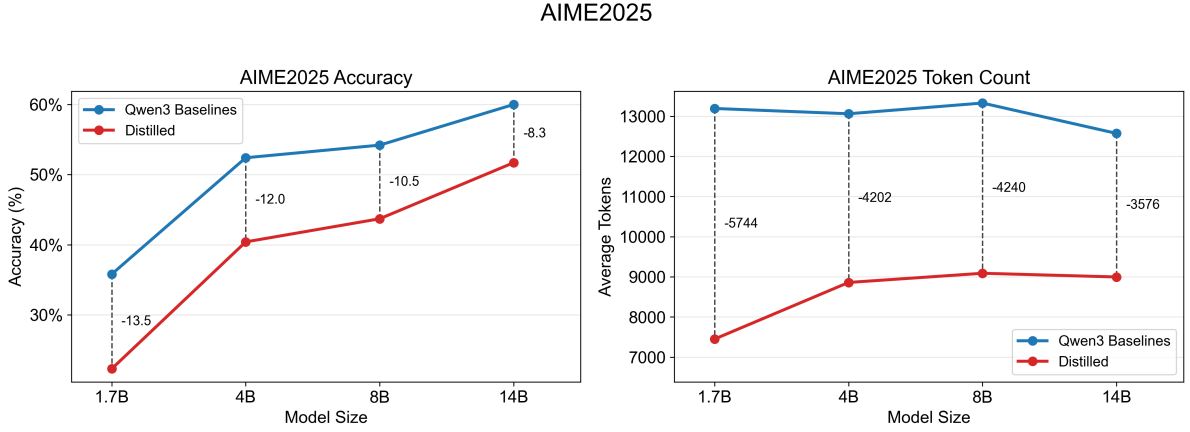


Figure 5.3.: AIME2025 success rate (Avg@4) and average trace tokens per generation for baseline versus distilled checkpoints across model sizes.

6. Conclusion

6.1. Summary

This thesis studies whether reasoning efficiency can be improved in a self-contained setting by having a model rewrite its own reasoning traces, and then distilling the resulting, more compute-optimal behavior, back into the model via token-level on-policy self-distillation. To evaluate rewriting beyond token counts, we introduce a benchmark that compares original and rewritten traces across correctness, compression, efficiency, step-level recall/precision against reference key steps, and reflection quality.

Empirically, self-refinement yields large reductions in reasoning trace length while preserving, and often improving, final-answer correctness across model sizes. Preliminary results show that on-policy distillation transfers a substantial portion of this conciseness into the student policy, with a slight drop in accuracy that is most visible on harder tasks. However, our results show that this gap reduces with model scale.

6.2. Limitations and Outlook

The reported distillation results are preliminary and should be interpreted as evidence that the proposed self-contained training signal can move the student distribution toward shorter traces, while leaving open how to most reliably preserve robustness on difficult tasks. A natural next step is to run our training loop on a dataset with more difficult and diverse problems. We hypothesize that if we cover a more general distribution of problems using our setup, we may overcome the performance drop on harder tasks. Orthogonally, future work involves refining how the rewrite-conditioned teacher context is constructed and used during training. For example, one can explore refreshing or diversifying per-instance static traces and strengthening the rewrite instruction to better preserve selective verification when needed. More broadly, using the benchmark diagnostics during training can help distinguish cases of productive compression from cases where the student becomes uniformly shorter in a way that harms hard-instance performance.

A. Prompt Templates

Rewriting Prompt Template

TASK

You will receive an input containing two parts: a **problem statement** and a **reasoning trace** (an authentic, detailed internal monologue of someone working through the problem).

Ask yourself: was this problem fundamentally easy, moderate, or hard for a careful human thinker? Based on that judgment, decide whether the original reasoning is *underthinking* (too sparse or rushed for a hard problem) or *overthinking* (unnecessarily elaborate for an easy problem).

Then let that judgment guide your edits:

- If the problem is easy and the trace is overthinking, your job is to carefully trim excess without flattening the voice or exploratory feel.
- If the problem is hard and the trace is underthinking, your job is to deepen the reasoning, slow it down, and add missing exploration.

Your job is to improve the reasoning trace WITHOUT destroying its voice. Your resulting reasoning process should explore the problem at a depth appropriate to its true difficulty, through a systematic long thinking process when needed.

THE ORIGINAL STYLE AND TONE MUST SURVIVE

This is your primary constraint. The original trace sounds like a person thinking out loud. Your reasoning must also sound like a person thinking out loud, the SAME person. If the original says "Wait, that can't be right..." your version should feel equally spontaneous and human. Never flatten exploratory language into formal explanation. Each step should include detailed considerations such as analyzing questions, brainstorming new ideas, verifying the accuracy of the current steps, refining any errors, and revisiting previous steps.

WHAT YOU MUST DO:

- FIX mistakes if you see clear logical errors and wrong assumptions
- KEEP productive tangents, self-corrections, and exploratory branches. These are features.
- INSERT missing logical steps. When there's a gap, add the missing reasoning AS IF the original thinker said it. Match their cadence and vocabulary.
- EXPAND compressed steps of the only reasoning by showing the work (calculations, sub-reasoning) inline, but write it how THEY would write it, not how a textbook would.
- DELETE only genuine redundancy that leads to nothing and does not contribute to the solution.

RESTRICTIONS:

- DO NOT rewrite sentences that are already clear
- DO NOT remove phrases like "hmm," "wait," "actually," "let me think..."
- DO NOT add formal transitions ("Furthermore," "In conclusion,")
- DO NOT convert the monologue into structured prose
- DO NOT summarize or compress unless the problem is clearly overthought for its difficulty

OPERATING PRINCIPLE:

Preserve and reuse original reasoning steps wherever possible. When you must add text, imitate the original's style closely. Explore several different paths of reasoning like the original if the problem difficulty demands that.


```
### OUTPUT RULES:
- After thinking, end with the closing thinking tag, then your output has to have a clear, step-by-
  step summary that walks through the key steps taken to arrive at the answer. This summary should
  :
  - Don't write step-by-step summary anywhere
  - Present the logical progression in clean, readable steps (not the raw internal monologue)
  - Conclude with the final answer in the same format as the original (e.g., \boxed{{}})
- **CRITICAL:** The final answer section must ALWAYS include a step-by-step summary before the boxed
  answer. NEVER jump straight from reasoning to just \boxed{{}} with no summary.

### THE INPUT FOLLOWS:
[Problem]
{problem}
[Reasoning Trace]
{student_trace}

Please reason step by step. Your new solution:
```

Find first occurrence of final answer prompt (Efficiency metric)

```
###TASK
You are given a problem, reasoning trace, and the model's final submitted answer.

Your task: Find the FIRST sentence in the reasoning trace where the model commits to the answer it
ultimately submits. This does NOT have to be the correct answer it is simply the answer the
model chose.

Rules for selecting the target sentence:
- It must be the EARLIEST sentence where the model's chosen answer value first appears as a
  conclusion.
- It may look like a direct result ("So x = 5"), a conclusion ("Therefore the answer is 5"), or a
  boxed statement.
- Do NOT select sentences that merely mention the answer value in passing (e.g., restating the
  problem or checking a different candidate). The sentence must be one where the model concludes
  or commits to that value.
- If no such sentence exists, copy the final sentence of the reasoning trace verbatim.

Output ONLY the following JSON object with no other text:
{{
  "first_sentence": "<copy the target sentence verbatim from the reasoning trace, character-for-
  character>"
}}

[Problem]
{problem}
[Model's Final Submitted Answer]
{final_answer}
[Reasoning]
{reasoning}
```

Multi-solution generation prompt

```
###TASK
You are an expert system that is presented with a question and a corresponding answer.
Generate up to 3 DIFFERENT solution approaches to reach the answer. Each approach must use genuinely
different mathematical techniques, logical paths, or problem decompositions.
List the key reasoning steps for each approach, making sure the steps are clear and complete.
You should pretend not to know the basic truth answer beforehand.

[Problem]
{problem}
[Answer]
{answer}

Output requirements:
1. Generate 1-3 genuinely different solution approaches (only include additional solutions if they
are truly different)
2. For each solution, record all essential key steps from start to answer
3. Only include the essential key steps, and don't output unnecessary words
4. Important: Output only JSON with no additional information. The output JSON must be valid at all
cost.

Output format
Here is the JSON output format:
[
  {{
    "solution1": {{
      "logical_conclusion": ["step1:", "step2:", "step3:", ...]
    }}
  }},
  {{
    "solution2": {{
      "logical_conclusion": ["step1:", "step2:", "step3:", ...]
    }}
  }}
]
```

Candidate reasoning step extraction prompt

```
### TASK
Extract and categorize reasoning steps from a candidate solution

Given a solution to a problem, extract each distinct reasoning step and categorize it.

## Step Categories:
1. **logical_inference**: Contains exactly one logical deduction that produces a new derived
conclusion. Must advance the solution.
2. **background_info**: External knowledge, definitions, restating given information, or initial
problem setup with no inference.
3. **other**: Summaries, transitions, meta-commentary, or steps that don't fit the above.

## Rules:
- Each step must be atomic (one conclusion per step)
- No content duplication across steps
- Preserve original order
- Maximum 35 steps
```

- Always include the final answer step
- Output ONLY valid JSON with no additional content

```
[Problem]
{question}
[Solution]
{solution}
```

```
## Output Format
[
  {{
    "step_number": 1,
    "description": "Step description",
    "category": "logical_inference|background_info|other"
  }}
]
```

Precision evaluation prompt

TASK

You are given a problem, a correct answer, ground truth key steps, and a list of pre-extracted reasoning steps from a candidate solution. Evaluate each step for correctness.

Evaluating Correctness

Apply the following decision procedure to every step, in order:

Step 1 Check for Match

A step is "Match" if ALL of the following hold:

- The step is a logical_inference (not background_info or other).
- Its conclusion either (a) appears verbatim as a ground truth step, OR (b) is directly and unambiguously entailed by one or more ground truth steps with no additional assumptions. "Entailed" means a competent mathematician would consider the conclusion a trivial consequence of the ground truth step(s) (e.g., algebraic rearrangement, unit conversion, substitution of a value already established in ground truth).

Step 2 Check for Reasonable (only if not Match)

A step is "Reasonable" if ALL of the following hold:

- The step is a logical_inference.
- Every premise the step relies on is either given in the problem or has already been established by a prior step that is itself Match or Reasonable.
- The logical form is valid (no non-sequiturs or hidden jumps).
- The conclusion does NOT contradict any ground truth step or the correct answer.

Note: "Reasonable" means the step could appear in a valid alternative solution path. It does NOT mean "plausible" or "common". If there is any doubt about whether a conclusion contradicts ground truth, assign "Wrong".

Step 3 Wrong

A step is "Wrong" if it is a logical_inference and fails either the Match or Reasonable tests i.e., it uses an invalid premise, contains a logical error, or its conclusion contradicts the ground truth or the correct answer.

Step 4 N/A

Assign "N/A" to every step whose step_type is "background_info" or "other". Do not assign Match/Reasonable/Wrong to those steps.

```
# Output Requirements
1. The output format MUST be in valid JSON format without ANY other content.
2. correct_answer: Whether the whole reasoning process produces the right answer.
```

```
## Output Format
[
  {{
    "step_index": <integer>,
    "step_type": "logical_inference|background_info|other",
    "judgment": "Match|Reasonable|Wrong|N/A",
    "correct_answer": "true|false"
  }}
]
```

```
Here is the problem, answer, ground truth, and the pre-extracted candidate steps to evaluate:
[Problem]
{problem}
[Correct Answer]
{answer}
[Ground Truth Key Steps]
{reference_key_steps}
[Candidate Steps]
{candidate_steps}
```

Recall evaluation prompt

```
### TASK
You are an expert system for verifying solutions. You are given multiple reference solutions (each
representing a different approach to solve the problem), and a candidate solution. Your tasks
are:
1. Determine which reference solution BEST matches the approach taken in the candidate solution
2. For that best-matching reference, evaluate each of its key steps against the candidate
```

```
# MATCHING PROCESS:
First, read the candidate solution and identify which reference solution approach it most closely
follows. Then, for the best-matching reference solution, evaluate each key step:
```

```
## Match Criteria:
- Exact Match or Equivalent Logical Step: A step is considered Matched if:
  - It appears exactly in the solution OR
  - The same logical reasoning or idea is clearly expressed, even if wording differs.
- Numerical and Conceptual Consistency: All key numbers, equations, or transformations should
  align.
- Step-by-Step Evaluation: Each key step must be assessed individually.
- Final Answer Check: Determine if the overall reasoning leads to the correct final answer.
```

```
# ADDITIONAL RULES:
1. Strict JSON Output: Output only the JSON object with no additional text.
2. No Omitted Steps: Every step of the best-matching reference solution must be evaluated.
```

```
Here is the problem, answer, candidate solution, and reference solutions:
[Problem]
{problem}
[Answer]
{answer}
[Candidate Solution]
```

```
{solution}
[Reference Solutions]
{reference_solutions}

# OUTPUT FORMAT:
{{
  "best_match_solution_id": "solution1|solution2|solution3",
  "step_judgments": [
    {{
      "step_index": <integer>,
      "judgment": "Matched" | "Unmatched",
      "correct_answer": "true" | "false"
    }}
  ]
}}
```

Reflection evaluation prompt

```
### TASK
Tag and evaluate reflection steps in a problem-solving reasoning trace.
Reflections are self-corrections or reconsiderations where the model doubts, revisits, or changes
its previous reasoning. They typically begin with phrases like:
- "But xxx"
- "Alternatively, xxx"
- "Maybe I should"
- "Let me double-check"
- "Wait xxx"
- "Perhaps xxx"
- "Actually xxx"
A reflection throws doubt on a previously reached conclusion or raises a new thought.

# Your Two Tasks:
## Task 1: Tag the Reasoning
Return the FULL chain of thought with every reflection wrapped in <reflection>...</reflection> tags.
A reflection includes everything from the doubt phrase until the next non-reflection sentence
begins. Do not modify any other content.

## Task 2: Evaluate each reflection
For each tagged reflection, output a JSON evaluation.

# Evaluation Criteria
Correct reflections must:
1. Reach accurate conclusions aligned with ground truth
2. Use new insights to find the mistake of the previous conclusion or verify its correctness.

Invalid reflections include:
1. Repetition - Restating previous content or method without new insights
2. Wrong Conclusion - Reaching incorrect conclusions vs ground truth
3. Incompleteness - Proposing but not executing new analysis methods
4. Other - Additional error types

# Input Format
[Problem]
{question}
[Chain of Thought]
{reasoning}
```

```
[Ground Truth Key Steps]
{reference_key_steps}

# Output Format
First, return the complete chain of thought with <reflection>...</reflection> tags inserted.
Then, on a new line after the tagged text, output ONLY the JSON array (one entry per reflection, in
order):
[
  {{
    "conclusion": "One-sentence summary of reflection outcome",
    "judgment": "Correct|Wrong",
    "error_type": "N/A|Repetition|Wrong Conclusion|Incompleteness|Other"
  }}
]
Rules:
1. Preserve original content and order
2. No new interpretations
3. Include ALL reflection steps
4. Empty JSON array [] if no reflections found
5. The tagged reasoning must come first, then the JSON array
```

A.1. Code availability

All code used to run the benchmark pipeline, generate the figures, and conduct the on-policy distillation experiments is available in the following public repositories:

- `rewritebench`: <https://github.com/yllkryeziu/rewritebench>
- `nvidia-rl` including on-policy self-distillation loop: <https://github.com/yllkryeziu/nvidia-rl>

List of Figures

- 5.1. Training dynamics for on-policy rewrite-distillation across Qwen3 model sizes.
Left: Training loss (EMA-smoothed, $\alpha=0.08$) **Center:** Validation accuracy on a held-out set of 128 OpenThoughts instances. **Right:** Average generation length on the validation set. 20
- 5.2. Math500 success rate (Avg@4) and average trace tokens per generation for baseline versus distilled checkpoints across model sizes. 21
- 5.3. AIME2025 success rate (Avg@4) and average trace tokens per generation for baseline versus distilled checkpoints across model sizes. 22

List of Tables

5.1. Benchmark results summary on the mathematics subset (1500 instances per model). Metrics are aggregated over instances and reported separately for original and rewritten traces.	18
5.2. On-policy distillation results on MATH500. We report success rate (Avg@4) and trace tokens per generation for the baseline and distilled checkpoints. Changes are computed as distilled minus baseline. Token limit for generations was 16384 tokens.	21
5.3. On-policy distillation results on AIME2025. We report success rate (Avg@4) and trace tokens per generation for the baseline and distilled checkpoints. Changes are computed as distilled minus baseline. Token limit for generations was 16384 tokens.	22

Bibliography

- [1] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. H. Chi, Q. V. Le, and D. Zhou. *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. 2022. doi: 10.48550/arXiv.2201.11903. arXiv: 2201.11903. URL: <https://arxiv.org/abs/2201.11903>.
- [2] DeepSeek-AI, D. Guo, D. Yang, H. Zhang, R. Xu, H. Zhu, R. Jia, Z. Zhang, X. Li, Q. Liu, W. Li, D. Ding, Z. Gao, Z. Wang, J. Chen, G. Li, Q. Zhang, S. Wu, C. Huang, X. Lin, Y. Hu, Z. Yang, Z. Xu, Q. Zhu, et al. “DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning”. In: *Nature* 645.8081 (2025). Also available as arXiv:2501.12948, pp. 633–638. ISSN: 1476-4687. doi: 10.1038/s41586-025-09422-z. arXiv: 2501.12948 [cs.CL]. URL: <https://arxiv.org/abs/2501.12948>.
- [3] OpenAI. *Learning to Reason with LLMs*. OpenAI Blog. 2024. URL: <https://openai.com/index/learning-to-reason-with-llms/> (visited on 02/28/2026).
- [4] X. Chen, J. Xu, T. Liang, Z. He, J. Pang, D. Yu, L. Song, Q. Liu, M. Zhou, Z. Zhang, R. Wang, Z. Tu, H. Mi, and D. Yu. *Do NOT Think That Much for 2+3=? On the Overthinking of o1-Like LLMs*. 2024. doi: 10.48550/arXiv.2412.21187. arXiv: 2412.21187. URL: <https://arxiv.org/abs/2412.21187>.
- [5] A. Lee, E. Che, and T. Peng. *How Well do LLMs Compress Their Own Chain-of-Thought? A Token Complexity Approach*. 2025. doi: 10.48550/arXiv.2503.01141. arXiv: 2503.01141. URL: <https://arxiv.org/abs/2503.01141>.
- [6] T. Han, Z. Wang, C. Fang, S. Zhao, S. Ma, and Z. Chen. *Token-Budget-Aware LLM Reasoning*. 2024. doi: 10.48550/arXiv.2412.18547. arXiv: 2412.18547. URL: <https://arxiv.org/abs/2412.18547>.
- [7] S. Xu, W. Xie, L. Zhao, and P. He. *Chain of Draft: Thinking Faster by Writing Less*. 2025. doi: 10.48550/arXiv.2502.18600. arXiv: 2502.18600. URL: <https://arxiv.org/abs/2502.18600>.
- [8] P. Aggarwal and S. Welleck. *L1: Controlling How Long a Reasoning Model Thinks with Reinforcement Learning*. 2025. doi: 10.48550/arXiv.2503.04697. arXiv: 2503.04697. URL: <https://arxiv.org/abs/2503.04697>.
- [9] N. Team. *Think Less, Achieve More: Cut Reasoning Costs by 50% Without Sacrificing Accuracy*. <https://novasky-ai.github.io/posts/reduce-overthinking>. Accessed: 2025-01-23. 2025.
- [10] Y. Meng, M. Xia, and D. Chen. *SimPO: Simple Preference Optimization with a Reference-Free Reward*. 2024. arXiv: 2405.14734 [cs.CL]. URL: <https://arxiv.org/abs/2405.14734>.

- [11] R. Agarwal, N. Vieillard, Y. Zhou, P. Stanczyk, S. Ramos, M. Geist, and O. Bachem. *On-Policy Distillation of Language Models: Learning from Self-Generated Mistakes*. 2023. DOI: 10.48550/arXiv.2306.13649. arXiv: 2306.13649. URL: <https://arxiv.org/abs/2306.13649>.
- [12] K. Lu and T. M. Lab. “On-Policy Distillation”. In: *Thinking Machines Lab: Connectionism* (2025). <https://thinkingmachines.ai/blog/on-policy-distillation>. DOI: 10.64434/tml.20251026.
- [13] G. Hinton, O. Vinyals, and J. Dean. *Distilling the Knowledge in a Neural Network*. 2015. DOI: 10.48550/arXiv.1503.02531. arXiv: 1503.02531. URL: <https://arxiv.org/abs/1503.02531>.
- [14] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelton, L. Miller, M. Simens, A. Askell, P. Welinder, P. Christiano, J. Leike, and R. Lowe. *Training language models to follow instructions with human feedback*. 2022. DOI: 10.48550/arXiv.2203.02155. arXiv: 2203.02155. URL: <https://arxiv.org/abs/2203.02155>.
- [15] S. Bengio, O. Vinyals, N. Jaitly, and N. Shazeer. *Scheduled Sampling for Sequence Prediction with Recurrent Neural Networks*. 2015. DOI: 10.48550/arXiv.1506.03099. arXiv: 1506.03099. URL: <https://arxiv.org/abs/1506.03099>.
- [16] Y. Kim and A. M. Rush. *Sequence-Level Knowledge Distillation*. 2016. DOI: 10.48550/arXiv.1606.07947. arXiv: 1606.07947. URL: <https://arxiv.org/abs/1606.07947>.
- [17] T. Furlanello, Z. C. Lipton, M. Tschannen, L. Itti, and A. Anandkumar. *Born Again Neural Networks*. 2018. DOI: 10.48550/arXiv.1805.04770. arXiv: 1805.04770. URL: <https://arxiv.org/abs/1805.04770>.
- [18] I. Shenfeld, M. Damani, J. Hübötter, and P. Agrawal. *Self-Distillation Enables Continual Learning*. 2026. arXiv: 2601.19897 [cs.LG]. URL: <https://arxiv.org/abs/2601.19897>.
- [19] R. Upadhayaya, M. R. Osti, Z. Smith, and C. Kottmyer. *Efficient LLM Context Distillation*. 2024. DOI: 10.48550/arXiv.2409.01930. arXiv: 2409.01930. URL: <https://arxiv.org/abs/2409.01930>.
- [20] T. Ye, L. Dong, X. Wu, S. Huang, and F. Wei. *On-Policy Context Distillation for Language Models*. 2026. DOI: 10.48550/arXiv.2602.12275. arXiv: 2602.12275. URL: <https://arxiv.org/abs/2602.12275>.
- [21] S. Ross, G. Gordon, and D. Bagnell. “A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning”. In: *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*. Vol. 15. Proceedings of Machine Learning Research. PMLR, 2011, pp. 627–635. URL: <https://proceedings.mlr.press/v15/ross11a.html>.
- [22] R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction*. 2nd ed. MIT Press, 2018. URL: <http://incompleteideas.net/book/the-book-2nd.html>.

- [23] T. Lei, R. Barzilay, and T. Jaakkola. *Rationalizing Neural Predictions*. 2016. DOI: 10.48550/arXiv.1606.04155. URL: <https://arxiv.org/abs/1606.04155>.
- [24] S. Yao, D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, and K. Narasimhan. *Tree of Thoughts: Deliberate Problem Solving with Large Language Models*. 2023. DOI: 10.48550/arXiv.2305.10601. URL: <https://arxiv.org/abs/2305.10601>.
- [25] M. Turpin, J. Michael, E. Perez, and S. Bowman. *Language Models Don't Always Say What They Think: Unfaithful Explanations in Chain-of-Thought Prompting*. 2023. DOI: 10.48550/arXiv.2305.04388. URL: <https://arxiv.org/abs/2305.04388>.
- [26] Anthropic. *Reasoning Models Don't Always Say What They Think*. 2025. DOI: 10.48550/arXiv.2505.05410. URL: <https://arxiv.org/pdf/2505.05410>.
- [27] Y. Wu, Y. Wang, T. Du, S. Jegelka, and Y. Wang. *When More is Less: Understanding Chain-of-Thought Length in LLMs*. 2025. DOI: 10.48550/arXiv.2502.07266. URL: <https://arxiv.org/abs/2502.07266>.
- [28] W. Yang, S. Ma, Y. Lin, and F. Wei. *Towards Thinking-Optimal Scaling of Test-Time Compute for LLM Reasoning*. 2025. DOI: 10.48550/arXiv.2502.18080. URL: <https://arxiv.org/abs/2502.18080>.
- [29] M. Ballon, A. Algaba, and V. Ginis. *The Relationship Between Reasoning and Performance in Large Language Models: o3 (mini) Thinks Harder, Not Longer*. 2025. DOI: 10.48550/arXiv.2502.15631. URL: <https://arxiv.org/abs/2502.15631>.
- [30] Y. Shen, J. Zhang, J. Huang, S. Shi, W. Zhang, J. Yan, N. Wang, K. Wang, and S. Lian. *DAST: Difficulty-Adaptive Slow-Thinking for Large Reasoning Models*. 2025. DOI: 10.48550/arXiv.2503.04472. URL: <https://arxiv.org/abs/2503.04472>.
- [31] J. Singh, J. C.-Y. Chen, A. Prasad, E. Stengel-Eskin, A. Nambi, and M. Bansal. *Think Right: Learning to Mitigate Under-Over Thinking via Adaptive, Attentive Compression*. 2025. DOI: 10.48550/arXiv.2510.01581. URL: <https://arxiv.org/abs/2510.01581>.
- [32] X. Wang, J. Wei, D. Schuurmans, Q. V. Le, E. H. Chi, S. Narang, A. Chowdhery, and D. Zhou. *Self-Consistency Improves Chain of Thought Reasoning in Language Models*. 2022. DOI: 10.48550/arXiv.2203.11171. URL: <https://arxiv.org/abs/2203.11171>.
- [33] C. Snell, J. Lee, K. Xu, and A. Kumar. "Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters". In: (2024). arXiv: 2408.03314 [cs.LG]. URL: <https://arxiv.org/abs/2408.03314>.
- [34] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegrefe, U. Alon, N. Dziri, S. Prabhume, Y. Yang, S. Gupta, B. P. Majumder, K. Hermann, S. Welleck, A. Yazdanbakhsh, and P. Clark. *Self-Refine: Iterative Refinement with Self-Feedback*. 2023. DOI: 10.48550/arXiv.2303.17651. URL: <https://arxiv.org/abs/2303.17651>.

- [35] Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, and I. Mordatch. “Improving Factuality and Reasoning in Language Models through Multiagent Debate”. In: (2023). arXiv: 2305.14325 [cs.CL]. URL: <https://arxiv.org/abs/2305.14325>.
- [36] L. Gao, A. Madaan, S. Zhou, U. Alon, P. Liu, Y. Yang, J. Callan, and G. Neubig. “PAL: Program-aided Language Models”. In: (2022). arXiv: 2211.10435 [cs.CL]. URL: <https://arxiv.org/abs/2211.10435>.
- [37] M. Damani, I. Shenfeld, A. Peng, A. Bobu, and J. Andreas. *Learning How Hard to Think: Input-Adaptive Allocation of LM Computation*. 2024. DOI: 10.48550/arXiv.2410.04707. arXiv: 2410.04707. URL: <https://arxiv.org/abs/2410.04707>.
- [38] J. Pan, Y. Zhang, C. Zhang, Z. Liu, H. Wang, and H. Li. *DynaThink: Fast or Slow? A Dynamic Decision-Making Framework for Large Language Models*. 2024. DOI: 10.48550/arXiv.2407.01009. arXiv: 2407.01009. URL: <https://arxiv.org/abs/2407.01009>.
- [39] R. Manvi, A. Singh, and S. Ermon. *Adaptive Inference-Time Compute: LLMs Can Predict if They Can Do Better, Even Mid-Generation*. 2024. DOI: 10.48550/arXiv.2410.02725. arXiv: 2410.02725. URL: <https://arxiv.org/abs/2410.02725>.
- [40] Y. Li, P. Yuan, S. Feng, B. Pan, X. Wang, B. Sun, H. Wang, and K. Li. *Escape Sky-High Cost: Early-Stopping Self-Consistency for Multi-Step Reasoning*. 2024. DOI: 10.48550/arXiv.2401.10480. arXiv: 2401.10480. URL: <https://arxiv.org/abs/2401.10480>.
- [41] X. Wang, S. Feng, Y. Li, P. Yuan, Y. Zhang, C. Tan, B. Pan, Y. Hu, and K. Li. *Make Every Penny Count: Difficulty-Adaptive Self-Consistency for Cost-Efficient Reasoning*. 2024. DOI: 10.48550/arXiv.2408.13457. arXiv: 2408.13457. URL: <https://arxiv.org/abs/2408.13457>.
- [42] M. Xu, Y. Li, K. Sun, and T. Qian. “Adaption-of-Thought: Learning Question Difficulty Improves Large Language Models for Reasoning”. In: *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 2024, pp. 5468–5495. URL: <https://aclanthology.org/2024.emnlp-main.313/>.
- [43] T. Munkhbat, N. Ho, S. Kim, Y. Yang, Y. Kim, and S.-Y. Yun. *Self-Training Elicits Concise Reasoning in Large Language Models*. 2025. DOI: 10.48550/arXiv.2502.20122. arXiv: 2502.20122. URL: <https://arxiv.org/abs/2502.20122>.
- [44] H. Luo, L. Shen, H. He, Y. Wang, S. Liu, W. Li, N. Tan, X. Cao, and D. Tao. *O1-Pruner: Length-Harmonizing Fine-Tuning for o1-Like Reasoning Pruning*. 2025. DOI: 10.48550/arXiv.2501.12570. arXiv: 2501.12570. URL: <https://arxiv.org/abs/2501.12570>.
- [45] X. Ma, G. Wan, R. Yu, G. Fang, and X. Wang. *CoT-Valve: Length-Compressible Chain-of-Thought Tuning*. 2025. DOI: 10.48550/arXiv.2502.09601. arXiv: 2502.09601. URL: <https://arxiv.org/abs/2502.09601>.
- [46] J. Zhang and C. Zuo. *GRPO-LEAD: A Difficulty-Aware Reinforcement Learning Approach for Concise Mathematical Reasoning in Language Models*. 2025. DOI: 10.48550/arXiv.2504.09696. arXiv: 2504.09696. URL: <https://arxiv.org/abs/2504.09696>.

- [47] J. Lee and J. Hockenmaier. *Evaluating Step-by-step Reasoning Traces: A Survey*. 2025. doi: 10.48550/arXiv.2502.12289. arXiv: 2502.12289. URL: <https://arxiv.org/abs/2502.12289>.
- [48] Z. Li, Y. Chang, and Y. Wu. *THINK-Bench: Evaluating Thinking Efficiency and Chain-of-Thought Quality of Large Reasoning Models*. 2025. doi: 10.48550/arXiv.2505.22113. arXiv: 2505.22113. URL: <https://arxiv.org/abs/2505.22113>.
- [49] E. Zelikman, Y. Wu, J. Mu, and N. D. Goodman. *STaR: Bootstrapping Reasoning With Reasoning*. 2022. doi: 10.48550/arXiv.2203.14465. arXiv: 2203.14465. URL: <https://arxiv.org/abs/2203.14465>.
- [50] Y. Huang, Y. Chen, Z. Yu, and K. McKeown. *In-context Learning Distillation: Transferring Few-shot Learning Ability of Pre-trained Language Models*. 2022. doi: 10.48550/arXiv.2212.10670. arXiv: 2212.10670. URL: <https://arxiv.org/abs/2212.10670>.
- [51] Kimi Team, Y. Bai, Y. Bao, Y. C. Chen, C. Chen, G. Chen, H. Chen, H. Chen, J. Chen, N. Chen, R. Chen, et al. *Kimi K2: Open Agentic Intelligence*. 2025. doi: 10.48550/arXiv.2507.20534. arXiv: 2507.20534 [cs.CL]. URL: <https://arxiv.org/abs/2507.20534>.
- [52] A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Gao, C. Huang, C. Lv, C. Zheng, D. Liu, F. Zhou, F. Huang, F. Hu, H. Ge, H. Wei, H. Lin, J. Tang, J. Yang, J. Tu, J. Zhang, J. Yang, J. Yang, J. Zhou, J. Zhou, J. Lin, K. Dang, K. Bao, K. Yang, L. Yu, L. Deng, M. Li, M. Xue, M. Li, P. Zhang, P. Wang, Q. Zhu, R. Men, R. Gao, S. Liu, S. Luo, T. Li, T. Tang, W. Yin, X. Ren, X. Wang, X. Zhang, X. Ren, Y. Fan, Y. Su, Y. Zhang, Y. Zhang, Y. Wan, Y. Liu, Z. Wang, Z. Cui, Z. Zhang, Z. Zhou, and Z. Qiu. *Qwen3 Technical Report*. 2025. arXiv: 2505.09388 [cs.CL]. URL: <https://arxiv.org/abs/2505.09388>.
- [53] D.-A. Team. *DeepSeek-V3.2: Pushing the Frontier of Open Large Language Models*. 2025. arXiv: 2512.02556 [cs.CL]. URL: <https://arxiv.org/abs/2512.02556>.
- [54] D. Jiang, R. Zhang, Z. Guo, Y. Li, Y. Qi, X. Chen, L. Wang, J. Jin, C. Guo, S. Yan, B. Zhang, C. Fu, P. Gao, and H. Li. *MME-CoT: Benchmarking Chain-of-Thought in Large Multimodal Models for Reasoning Quality, Robustness, and Efficiency*. 2025. arXiv: 2502.09621 [cs.CV]. URL: <https://arxiv.org/abs/2502.09621>.
- [55] F. Srambical. “Going Beyond the Causal Mask in Language Modeling”. In: *p(doom) blog* (2024). <https://pdoom.org/blog.html>.
- [56] Open Thoughts Team. *open-thoughts/OpenThoughts-114k*. Hugging Face dataset card. 2025. URL: <https://huggingface.co/datasets/open-thoughts/OpenThoughts-114k> (visited on 02/18/2026).
- [57] Alibaba ModelScope. *Evaluating the Qwen3 Model*. EvalScope documentation (Read the Docs). Accessed: 2026-03-02. URL: https://evalscope.readthedocs.io/en/latest/best_practice/qwen3.html.
- [58] I. Loshchilov and F. Hutter. “Decoupled Weight Decay Regularization”. In: *International Conference on Learning Representations*. 2019. URL: <https://openreview.net/forum?id=Bkg6RiCqY7>.